OneKhusa Laravel Integration Reference

Hosted Checkout Flow & Webhook Automation

Onekhusa Team

May 2026

1 Introduction

This document serves as a comprehensive guide for developers integrating OneKhusa into a Laravel environment. This implementation utilizes Laravel's powerful Service Layer and HTTP Client (Guzzle wrapper) to manage the payment lifecycle. Due to PHP's request-response lifecycle, a **Status Polling** strategy is implemented on the frontend to synchronize the UI with backend webhook updates.

2 Project Directory Structure

The project adheres to Laravel's standard MVC architecture, with a dedicated Service class for external API logic.

```
onekhusa-laravel-integration/
├── .env ..... Sensitive Credentials
├── bootstrap/
│   └── app.php ..... Middleware & CSRF config
├── app/
│   ├── Http/Controllers/
│   │   ├── TicketController.php ..... Purchase & Status Logic
│   │   └── WebhookController.php ..... OneKhusa Callbacks
│   └── Services/
│       └── OneKhusaService.php ..... API Guzzle Client
├── resources/views/
│   └── welcome.blade.php ..... Fintech Dashboard
└── routes/
    ├── api.php ..... Backend Endpoints
    └── web.php ..... Frontend Routes
```

3 Environment Configuration (.env)

All integration keys must be defined in the `.env` file. The `PUBLIC_CALLBACK_URL` must match the active NGrok tunnel during development.

```
ONEKHUSA_API_KEY=sandbox_vDdLumyf...
ONEKHUSA_API_SECRET=_fefXaN9LGorR...
ONEKHUSA_ORG_ID=0BBNREQ33RSX
ONEKHUSA_MERCHANT_NUMBER=79619974

ONEKHUSA_CHECKOUT_URL=https://api.onekhusa.com/sandbox/v1/checkout/rtp/initiate
PUBLIC_CALLBACK_URL=https://zena-unjudgeable-renita.ngrok-free.dev
```

Listing 1: .env Configuration

4 Core Service Layer

The service class encapsulates the API logic, ensuring the `X-Idempotency-Key` is unique for every session.

```
public function initiateCheckout($amount, $reference) {
    $payload = [
        "authentication" => ["apiKey" => env('API_KEY'), "apiSecret" => env('
    API_SECRET')],
        "merchant" => ["organisationId" => env('ORG_ID'), "merchantAccountNumber" =>
    (int)env('MERCHANT_NO')],
        "payment" => ["sourceReferenceNumber" => $reference, "amount" => (float)
    $amount],
        "route" => [
            "successRedirectionUrl" => env('CALLBACK_URL') . "/?ref=" . $reference,
            "callbackApiUrl" => env('CALLBACK_URL') . "/api/webhooks/payments"
        ]
    ];

    $response = Http::withHeaders([
        'X-Idempotency-Key' => 'PHP-CHK-' . $reference . '-' . Str::random(8)
    ])->post(env('CHECKOUT_URL'), $payload);

    return $response->json();
}
```

Listing 2: app/Services/OneKhusaService.php

5 Webhook & CSRF Security

OneKhusa sends notifications as `POST` requests without CSRF tokens. In Laravel 11, the callback path must be exempted in `bootstrap/app.php`.

```
->withMiddleware(function (Middleware $middleware) {
    $middleware->validateCsrfTokens(except: [
        'api/webhooks/payments',
    ]);
})
```

Listing 3: bootstrap/app.php Exemption

6 Transaction Synchronization

The backend uses **Laravel's Cache** to store the status of temporary references. This allows the frontend to poll for the "Paid" state after the user is redirected back from OneKhusa.

```
public function handle(Request $request) {
    $payload = $request->all();
    $myRef = $payload['metaData']['ReferenceNumber'] ?? $payload['
    sourceReferenceNumber'];

    if ($payload['responseCode'] === "S100" || $payload['transactionStatusCode'] ===
    "S") {
        Cache::put("status_$myRef", "Paid", 1800); // Store for 30 mins
    }
    return response("acknowledged", 200)->header('Content-Type', 'text/plain');
}
```

Listing 4: Webhook Logic

7 Frontend: Verification Overlay

The dashboard utilizes a JavaScript-based overlay. Upon redirect landing (detected via the `ref` URL parameter), the UI polls the `api/Tickets/status/{ref}` endpoint every 3 seconds until confirmation is received.

8 Conclusion

The Laravel implementation provides an enterprise-ready approach to OneKhusa integration. By leveraging Service Providers, Cache-based status tracking, and CSRF exemptions, developers can create a seamless and secure checkout experience that adheres to modern PHP best practices.